



Understanding AWT Layout Managers

LAYOUT Manager

A layout manager automatically arranges

- your controls within a window
- by using some type of algorithm.

If you have programmed for other GUI environments,

- such as Windows,
- then you are accustomed to laying out your controls by hand

LAYOUT Manager

Each **Container** object has a

- layout manager associated with it.

A layout manager is an instance of any class

- that implements the **LayoutManager** interface.

The layout manager is

- set by the **setLayout()** method.

If no call to **setLayout()** is made,

- then the default layout manager is used.

LAYOUT Manager

Whenever a container is resized

- (or sized for the first time),
- the layout manager is used to position each of the components within it.

The **setLayout()** method has the following general form:

- void **setLayout**(LayoutManager *layoutObj*)

Here, *layoutObj* is a reference to

- the desired layout manager

AWT Layout Manager Classes

1.

- **Border Layout**
- The `BorderLayout` arranges the components to fit in the five regions: east, west, north, south and center

2

- **CardLayout**
- The `CardLayout` object treats each component in the container as a card. Only one card is visible at a time.

3

- **FlowLayout**
- The `FlowLayout` is the default layout. It layouts the components in a directional flow.

4

- **GridLayout**
- The `GridLayout` manages the components in form of a rectangular grid.

AWT Layout Manager Classes

▸ Border Layout



▸ CardLayout



AWT Layout Manager Classes

▸ Flow Layout



▸ CardLayout



Flow Layout

FlowLayout is the

- default layout manager.
- This is the layout manager that the preceding examples have used.

FlowLayout
implements a simple
layout style,

- which is similar to how words flow in a text editor.

The direction of the
layout is

- governed by the container's component orientation property,
- which, by default, is left to right, top to bottom.



Flow Layout

Flow Layout

Therefore, by default,

- components are laid out line-by-line
- beginning at the upper-left corner.

In all cases, when a line is filled,

- layout advances to the next line.

A small space is left between each component,

- above and below,
- as well as left and right.

FlowLayout (Constructors)

FlowLayout():

- This creates the default layout,
- which centers components and leaves five pixels of space between each component

FlowLayout(int *how*) :

- This form lets you specify how each line is aligned. Valid values for *how* are as follows:
 - FlowLayout.LEFT
 - FlowLayout.CENTER
 - FlowLayout.RIGHT
 - FlowLayout.LEADING
 - FlowLayout.TRAILING

FlowLayout(int *how*, int *horz*, int *vert*) :

- This constructor allows you to specify the **horizontal** and **vertical** space left
- between components in *horz* and *vert*, respectively.

Border Layout

The **BorderLayout** class implements a

- common layout style for top-level windows.

It has four narrow, fixed-width components

- at the edges and one large area in the center.

The four sides are referred to as north, south, east, and west.

- The middle area is called the center.

Here are the constructors defined by **BorderLayout**:

BorderLayout() :

- This form creates a default border layout

BorderLayout(int *horz*, int *vert*) :

- This form allows you to specify the horizontal and vertical space left between components in ***horz*** and ***vert***, respectively.



Border Layout

Border Layout

BorderLayout.NORTH

BorderLayout.SOUTH

BorderLayout.CENTER

BorderLayout.EAST

BorderLayout.WEST

Border Layout

When adding components,

- you will use these constants with the following form of **add()**,
- which is defined by **Container**:

```
void add(Component compObj, Object  
region)
```

- Here, *compObj* is the component to be added, and *region* specifies
- where the component will be added.

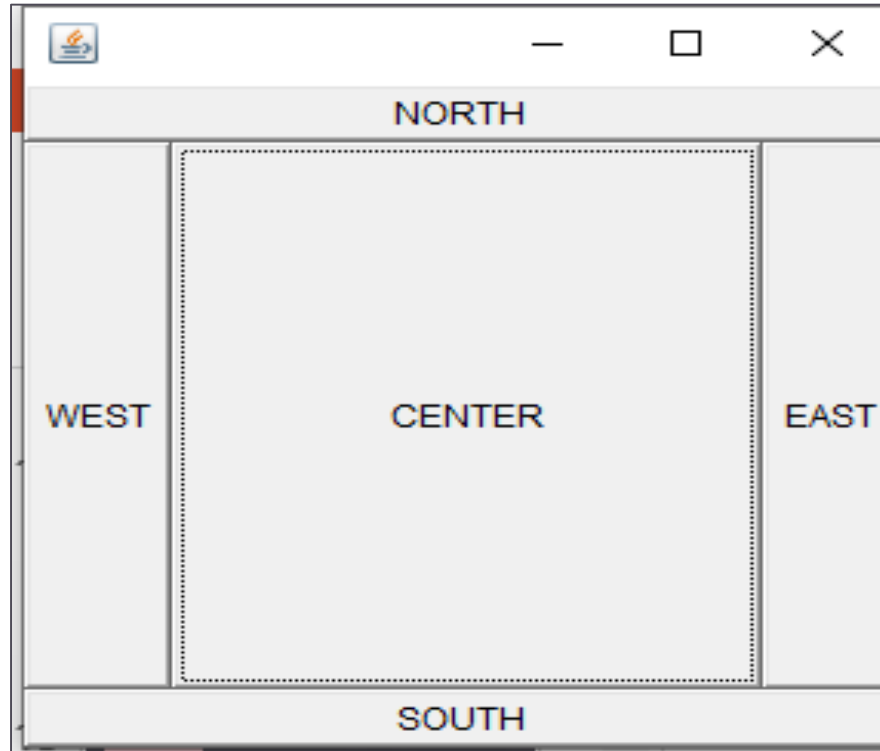
Border Layout

```
change.log x ButtonExample.java x Border_Layout.java x
1  import java.awt.*;
2  import java.awt.event.*;
3  public class Border_Layout
4  {
5      Frame f;
6      Border_Layout()
7      {
8          f = new Frame();
9          // creating buttons
10         Button b1 = new Button("NORTH");; // the button will be labeled as NORTH
11         Button b2 = new Button("SOUTH");; // the button will be labeled as SOUTH
12         Button b3 = new Button("EAST");; // the button will be labeled as EAST
13         Button b4 = new Button("WEST");; // the button will be labeled as WEST
14         Button b5 = new Button("CENTER");; // the button will be labeled as CENTER
15     }
```

Border Layout

```
16 f.add(b1, BorderLayout.NORTH); // b1 will be placed in the North Direction
17 f.add(b2, BorderLayout.SOUTH); // b2 will be placed in the South Direction
18 f.add(b3, BorderLayout.EAST); // b2 will be placed in the East Direction
19 f.add(b4, BorderLayout.WEST); // b2 will be placed in the West Direction
20 f.add(b5, BorderLayout.CENTER); // b2 will be placed in the Center
21
22 f.setSize(300, 300);
23 f.setVisible(true);
24 }
25 public static void main(String[] args) {
26     new Border_Layout();
27 }
```

Border Layout: OUTPUT





Grid Layout

Grid Layout



GridLayout lays out components in a two-dimensional grid.



When you instantiate a **GridLayout**,

- you define the number of rows and columns.

Grid Layout Constuctor

GridLayout() :

- This form creates a single-column grid layout.

GridLayout(int numRows, int numColumns) :

- This form creates a grid layout with the specified number of rows and columns.

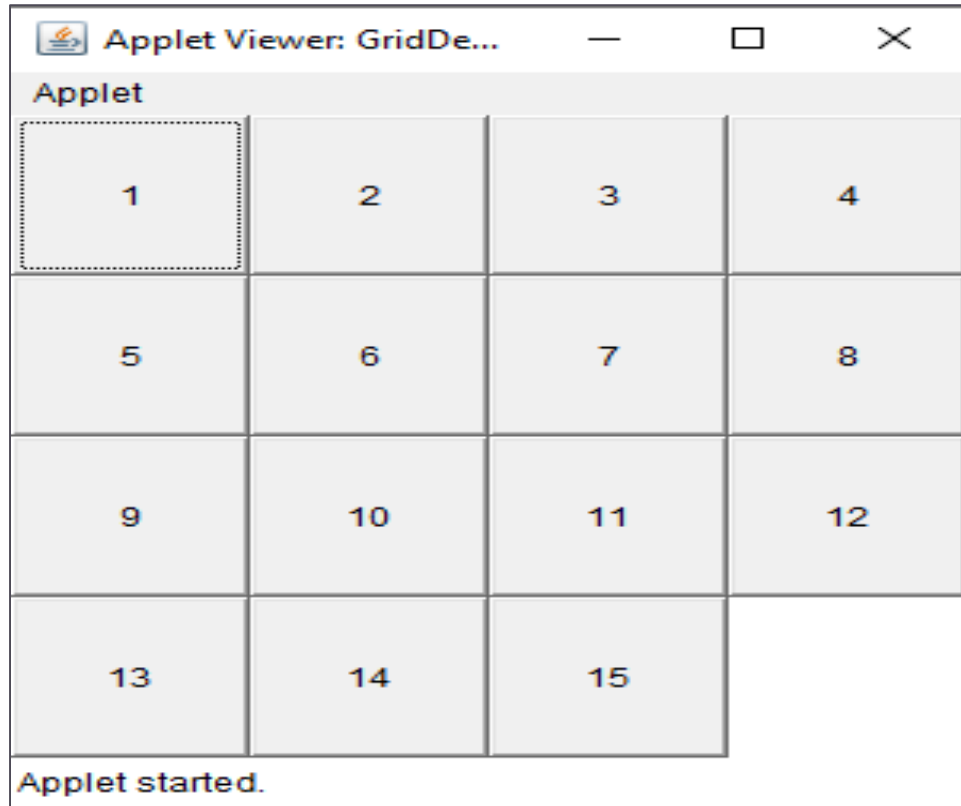
GridLayout(int numRows, int numColumns, int horz, int vert) :

- This form allows you to specify the horizontal and vertical space left between components in *horz* and *vert*, respectively.
- Either *numRows* or *numColumns* can be zero.
- Specifying *numRows* as zero allows for unlimited-length columns.
- Specifying *numColumns* as zero allows for unlimited-length rows.

Grid Layout

```
Border_Layout.java x GridDemo.java x
1 E:\HPC\Source\Java\AWT\GridDemo\Prog\AWT\Border_Layout.java
2 import java.awt.*;
3 import java.applet.*;
4 /* <applet code="GridDemo.class" width=300 height=300>
5 </applet>*/
6 public class GridDemo extends Applet
7 {
8     static final int n = 4;
9     public void init()
10    {
11        setLayout(new GridLayout(n, n));
12        for(int i = 0; i < n; i++)
13        {
14            for(int j = 0; j < n; j++)
15            {
16                int k = i * n + j;
17                if(k > 0)
18                    add(new Button("" + k));
19            }
20        }
21    }
```

Grid Layout: OUTPUT





Card Layout

Card Layout

The **CardLayout** class is unique

- among the other layout managers in that
- it stores several different layouts.

Each layout can be thought of

- as being on a separate index card in a deck
- that can be shuffled so that any card is on top at a given time.

This can be useful for user

- interfaces with optional components that can be dynamically enabled
- and disabled upon user input.

You can prepare the other layouts and have them hidden,

- ready to be activated when needed.

Card Layout Constructor

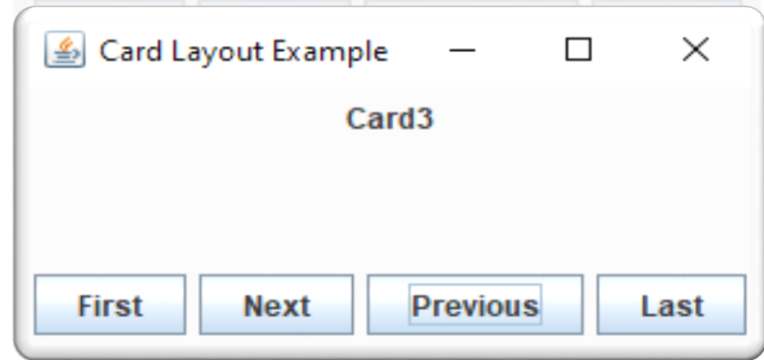
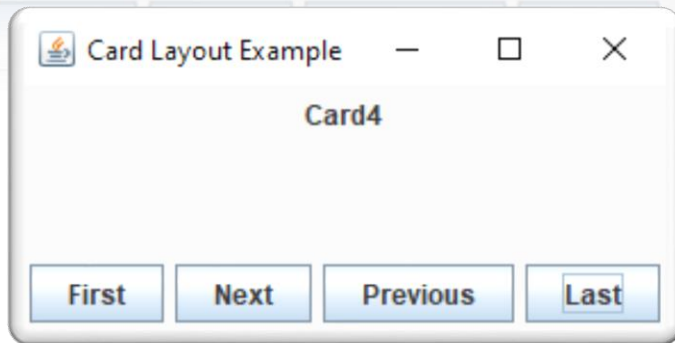
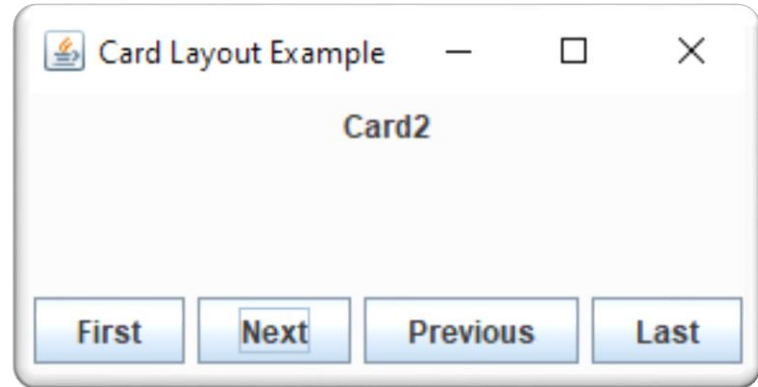
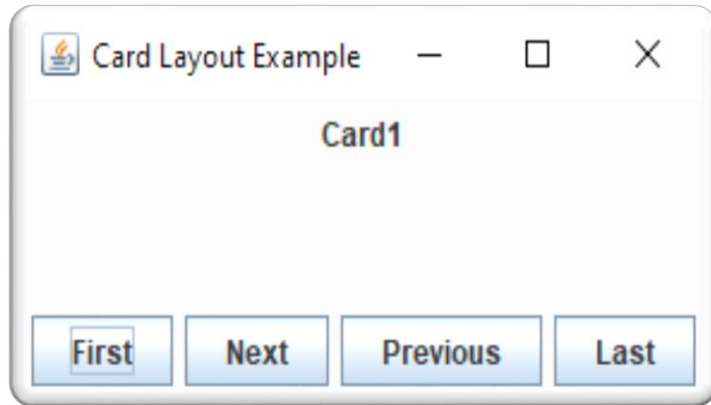
CardLayout provides these two constructors:

CardLayout() : This form creates a default card layout.

CardLayout(int *horz*, int *vert*) :

- This form allows you to specify the
- horizontal and vertical space left
- between components in ***horz*** and ***vert***, respectively.

Card Layout: **OUTPUT**





Box Layout

Box Layout

**The Java
BoxLayout class
is used**

- to arrange the components either
- vertically or horizontally.

**For this purpose,
the BoxLayout
class provides**

- four constants.

*Fields
of
BoxLayout
Class*

public static final int X_AXIS:

- Alignment of the components are horizontal from left to right.

public static final int Y_AXIS:

- Alignment of the components are vertical from top to bottom.

Fields of BoxLayout Class

public static final int LINE_AXIS:

Alignment of the components is similar to the way words are aligned in a line, which is based on the `ComponentOrientation` property of the container.

If the `ComponentOrientation` property of the container is horizontal, then the components are aligned horizontally; otherwise, the components are aligned vertically.

For horizontal orientations, we have two cases: left to right, and right to left. If the value `ComponentOrientation` property of the container is from left to right, then the components are rendered from left to right, and for right to left, the rendering of components is also from right to left.

In the case of vertical orientations, the components are always rendered from top to bottom

Fields of BoxLayout Class

```
public static  
final int  
PAGE_AXIS:
```

- Alignment of the components is similar to the way text lines are put on a page, which is based on the `ComponentOrientation` property of the container.
- If the `ComponentOrientation` property of the container is horizontal, then components are aligned vertically; otherwise, the components are aligned horizontally.
- For horizontal orientations, we have two cases: left to right, and right to left. If the value `ComponentOrientation` property of the container is also from left to right, then the components are rendered from left to right, and for right to left, the rendering of components is from right to left.
- In the case of vertical orientations, the components are always rendered from top to bottom.

***Box
Layout
Constructor***

**BoxLayout(Container c, int
axis):**

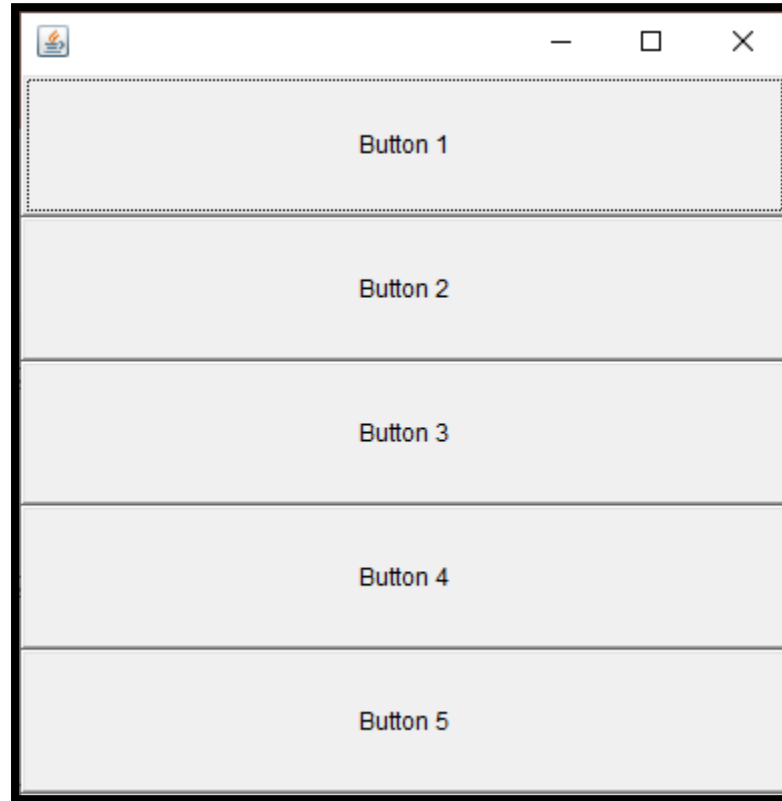
creates a box
layout that
arranges the
components

with the
given axis.

Example of BoxLayout class with Y-AXIS

```
BoxLayoutExample1.java
1  import java.awt.*;    import javax.swing.*;
2  public class BoxLayoutExample1 extends Frame
3  {
4      Button buttons[];
5      public BoxLayoutExample1()
6      {
7          buttons = new Button [5];
8          for (int i = 0; i < 5; i++)
9          {
10             buttons[i] = new Button ("Button " + (i + 1));
11             // adding the buttons so that it can be displayed
12             add (buttons[i]);
13         }
14         // the buttons will be placed horizontally
15         setLayout (new BoxLayout (this, BoxLayout.Y_AXIS));
16         setSize (400, 400);
17         setVisible (true);
18     }
19     public static void main (String args[])
20     {
21         BoxLayoutExample1 b = new BoxLayoutExample1 ();
22     }
```

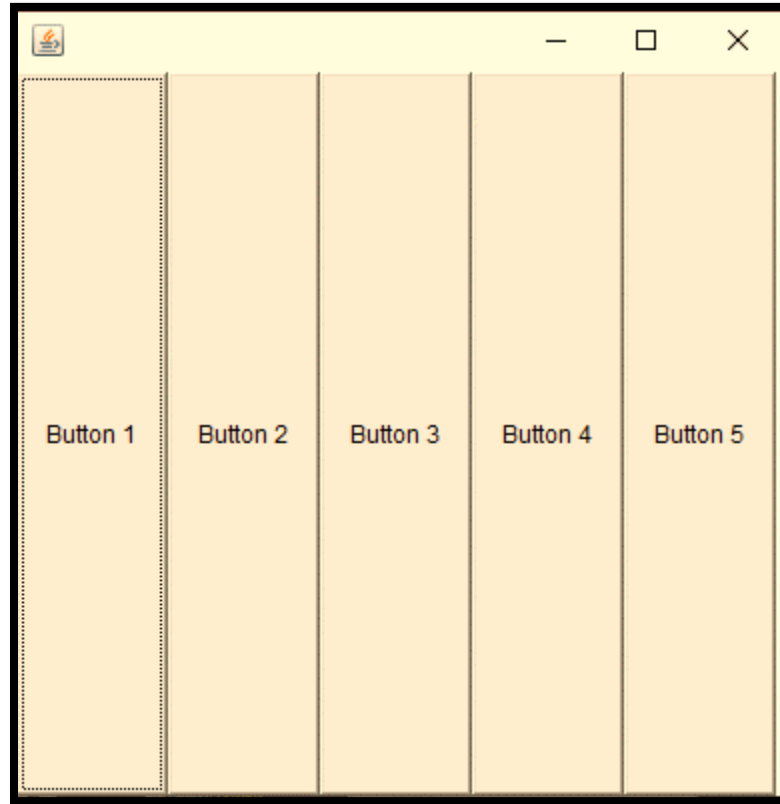
Box Layout1: OUTPUT



Example of BoxLayout class with X-AXIS

```
BoxLayoutExample2.java
1  import java.awt.*;
2  import javax.swing.*;
3  public class BoxLayoutExample2 extends Frame
4  {
5      Button buttons[];
6      public BoxLayoutExample2 ()
7      {
8          buttons = new Button [5];
9          for (int i = 0; i < 5; i++)
10         {
11             buttons[i] = new Button ("Button " + (i + 1));
12             add (buttons[i]);
13         }
14         // the buttons in the output will be aligned vertically
15         setLayout (new BoxLayout (this, BoxLayout.X_AXIS));
16         setSize (400, 400);
17         setVisible (true);
18     }
19     public static void main (String args[])
20     {
21         BoxLayoutExample2 b = new BoxLayoutExample2 ();
22     }
```

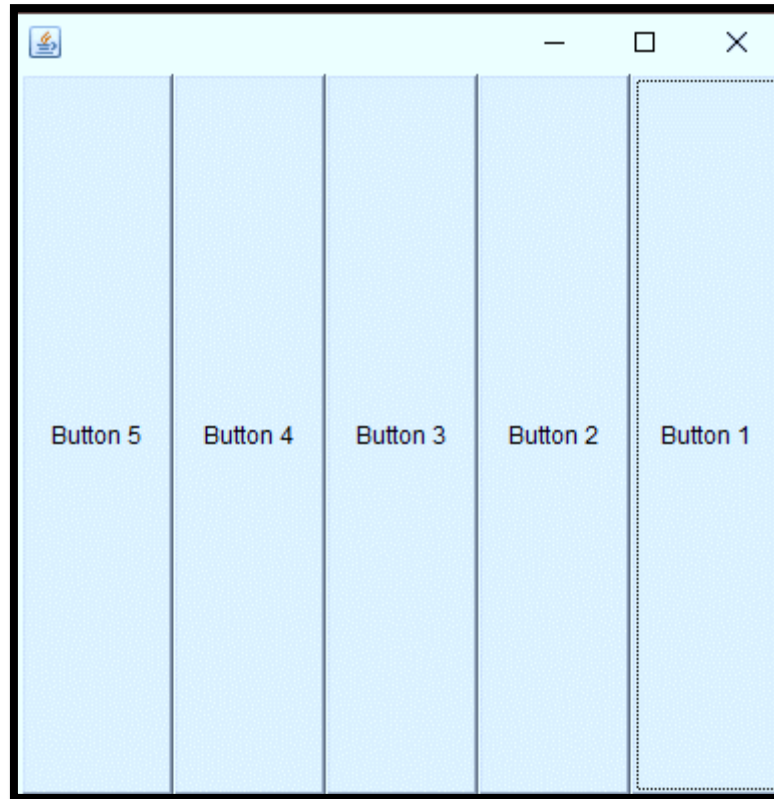

Box Layout2: OUTPUT



Example of BoxLayout class with Line-AXIS

```
BoxLayoutExample3.java
1  import java.awt.*;   import javax.swing.*;
2  public class BoxLayoutExample3 extends Frame
3  {
4      Button buttons[];
5      public BoxLayoutExample3 ()
6      {
7          buttons = new Button[5];
8          setComponentOrientation(ComponentOrientation.RIGHT_TO_LEFT);
9          for (int i = 0; i < 5; i++)
10         {
11             buttons[i] = new Button ("Button " + (i + 1));
12             add (buttons[i]);
13         }
14         // the ComponentOrientation is set to RIGHT_TO_LEFT. Therefore,
15         // the added buttons will be rendered from right to left
16         setLayout (new BoxLayout(this, BoxLayout.LINE_AXIS));
17         setSize (400, 400);
18         setVisible(true);
19     }
20     public static void main (String args[])
21     {
22         BoxLayoutExample3 b=new BoxLayoutExample3 (); } }
```

Box Layout3: OUTPUT





GridBagLayout

GridBag Layout

The Java
GridBagLayout class is
used

- to align components vertically, horizontally or along their baseline.

The components may
not be of the

- same size.

Each GridBagLayout
object maintains a

- dynamic, rectangular grid of cells.

Each component
occupies

- one or more cells known as its display area.

Each component
associates an

- instance of
GridBagConstraints.

With the help of the
constraints object,

- we arrange the component's display area on the grid.

GridBag Layout

The GridBagLayout manages

- each component's minimum and preferred sizes in order to determine the component's size.

GridBagLayout components are also arranged

- in the rectangular grid
- but can have many different sizes and
- can occupy multiple rows or columns.

***GridBag
Layout
Constructor***

GridBagLayout():

**The
parameterless
constructor is
used**

**to create a
grid bag
layout
manager.**

*Commonly
Used
Methods:*

removeLayoutComponent(Component cmp):

- Removes the specified component from this layout.

getLayoutAlignmentY(Container p):

- Returns the alignment along the y-axis.

addLayoutComponent(Component cmp, Object cons):

- Add the specified component with the specified name to the layout.

toString():

- Returns a string representation of this grid bag layout's values.

*Commonly
Used
Methods:*

getLayoutAlignmentX(Container p):

- Returns the alignment along the x-axis.

getConstraints(Component cmp):

- Gets the constraints for the specified component.

maximumLayoutSize(Container tar):

- Returns the maximum dimensions for this layout given the components in the specified target container.

minimumLayoutSize(Container par):

- Determines the minimum size of the parent container using this grid bag layout.

Instance variables to manipulate the GridBagLayout object Constraints:

Variable	Role
gridx and gridy	• These contain the coordinates of the origin of the grid. • They allow a at a specific position of a component positioning. • By default, they have GrigBagConstraint.RELATIVE value which indicates that a component can be stored to the right of previous
gridwidth, gridheight	• Define how many cells will occupy component (height and width). by The default is 1. • The indication is relative to the other components of the line or the column. • The GridBagConstraints.REMAINDER value specifies that the next component inserted will be the last of the line or the current column. • the value GridBagConstraints.RELATIVE up the component after the last component of a row or column.
fill	• Defines the fate of a component smaller than the grid cell. GridBagConstraints .NONE retains the original size: Default GridBagConstraints. • HORIZONTAL expanded horizontally GridBagConstraints. • VERTICAL GridBagConstraints.BOTH expanded vertically expanded to the dimensions of the cell

Instance variables to manipulate the GridBagLayout object Constraints:

Variable	Role
ipadx, ipady	Used to define the horizontal and vertical expansion of components. not works if expansion is required by fill. The default value is (0,0).
anchor	When a component is smaller than the cell in which it is inserted, it can be positioned using this variable to define the side from which the control should be aligned within the cell. Possible variables NORTH, NORTHWEST, NORTHEAST, SOUTH, SOUTHWEST, SOUTHEAST, WEST and EAST
weightx, weighty	Used to define the distribution of space in case of change of dimension

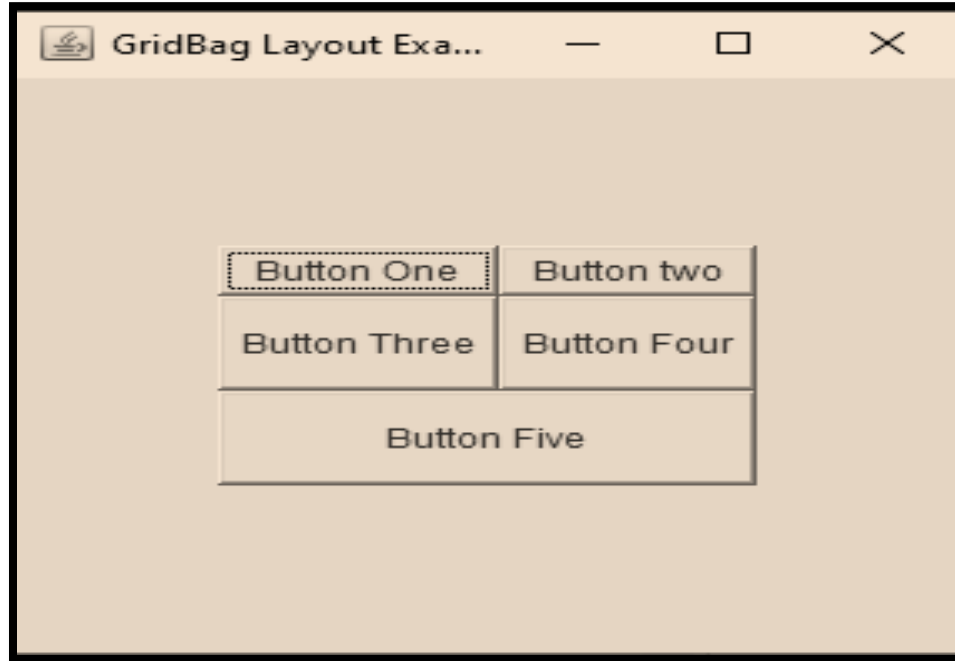
Example of GridBagLayout

```
GridBagLayoutExample.java
1  import java.awt.Button;
2  import java.awt.GridBagConstraints;
3  import java.awt.GridBagLayout;
4  import javax.swing.*;
5  public class GridBagLayoutExample extends JFrame
6  {
7      public static void main(String[] args)
8      {
9          GridBagLayoutExample a = new GridBagLayoutExample();
10         public GridBagLayoutExample()
11         {
12             GridBagLayout grid = new GridBagLayout();
13             GridBagConstraints gbc = new GridBagConstraints();
14             setLayout(grid);
15             setTitle("GridBag Layout Example");
16             GridBagLayout layout = new GridBagLayout();
17             this.setLayout(layout);
18             gbc.fill = GridBagConstraints.HORIZONTAL;
19             gbc.gridx = 0;
20             gbc.gridy = 0;
21             this.add(new Button("Button One"), gbc);
```

Example of GridBagLayout

```
GridBagLayoutExample.java x
22     gbc.gridx = 1;
23     gbc.gridy = 0;
24     this.add(new Button("Button two"), gbc);
25     gbc.fill = GridBagConstraints.HORIZONTAL;
26     gbc.ipady = 20;
27     gbc.gridx = 0;
28     gbc.gridy = 1;
29     this.add(new Button("Button Three"), gbc);
30     gbc.gridx = 1;
31     gbc.gridy = 1;
32     this.add(new Button("Button Four"), gbc);
33     gbc.gridx = 0;
34     gbc.gridy = 2;
35     gbc.fill = GridBagConstraints.HORIZONTAL;
36     gbc.gridwidth = 2;
37     this.add(new Button("Button Five"), gbc);
38         setSize(300, 300);
39         setPreferredSize(getSize());
40         setVisible(true);
41         setDefaultCloseOperation(EXIT_ON_CLOSE);
42     }
43 }
```

GridBag Layout Example1: OUTPUT





Group Layout

Group Layout

GroupLayout

- *groups its components and*
- *places them in a Container hierarchically.*
- The grouping is done by instances of the Group class.

Group is an abstract class, and

- two concrete classes
- which implement this Group class are
- SequentialGroup and ParallelGroup.

Group Layout

SequentialGroup positions

- its child sequentially one after another
- whereasParallelGroup
- aligns its child on top of each other.

The GroupLayout class provides methods such as

- createParallelGroup()
and
- createSequentialGroup()
to create groups.

Group Layout

GroupLayout treats each axis independently.

- there is a group representing the horizontal axis, and
- a group representing the vertical axis.

Each component must exist in

- both a horizontal and vertical group,
- otherwise an `IllegalStateException` is thrown during layout or
- when the minimum, preferred, or maximum size is requested.

Group Layout

Class	Description
GroupLayout.Align ment	Enumeration of the possible ways GroupLayout can align its children.
GroupLayout.Group	Group provides the basis for the two types of operations supported by GroupLayout: laying out components one after another (SequentialGroup) or aligned (ParallelGroup).
GroupLayout.Parall elGroup	It is a Group that aligns and sizes it's children.
GroupLayout.Sequ entialGroup	It is a Group that positions and sizes its elements sequentially, one after another.

Commonly Used Methods:

`addLayoutComponent(Component component, Object constraints)`

- It notify that a Component has been added to the parent container.

`addLayoutComponent(String name, Component component)`

- It notify that a Component has been added to the parent container.

`createBaselineGroup(boolean resizable, boolean
anchorBaselineToTop)`

- It creates and returns a `ParallelGroup` that aligns it's elements along the baseline.

`createParallelGroup()`

- It creates and returns a `ParallelGroup` with an alignment of `Alignment.LEADING`

***Commonly
Used
Methods:***

`createParallelGroup(GroupLayout.Alignment alignment)`

- It creates and returns a `ParallelGroup` with the specified alignment.

`createSequentialGroup()`

- It creates and returns a `SequentialGroup`.

`getAutoCreateContainerGaps()`

- It returns true if gaps between the container and components that border the container are automatically created.

`getAutoCreateGaps()`

- It returns true if gaps between components are automatically created.

***Commonly
Used
Methods:***

getHonorsVisibility()

- It returns whether component visibility is considered when sizing and positioning components.

getLayoutAlignmentX(Container parent)

- It returns the alignment along the x axis.

getLayoutAlignmentY(Container parent)

- It returns the alignment along the y axis.

maximumLayoutSize(Container parent)

- It returns the maximum size for the specified container.

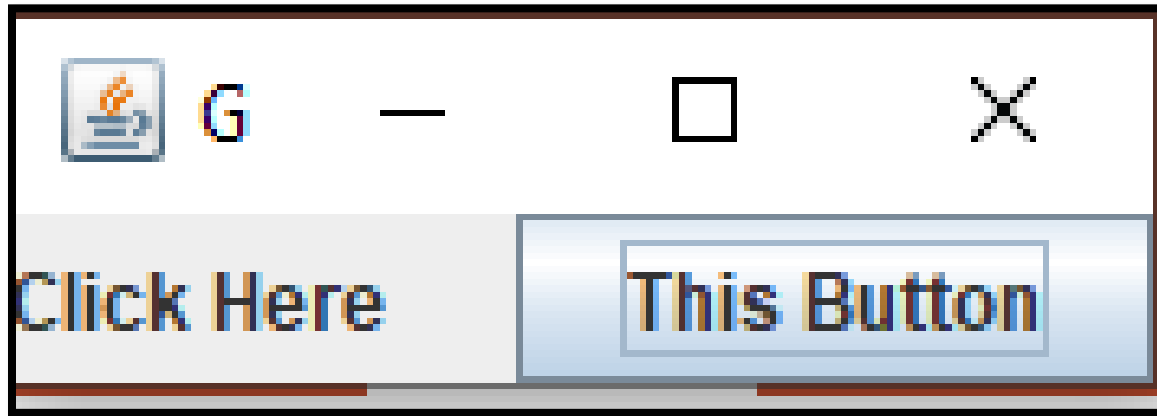
Group Example1:

```
GroupExample.java x GridBagLayoutDemo.java x
1  import java.awt.*;
2  import javax.swing.*;
3  public class GroupExample {
4      public static void main(String[] args) {
5          JFrame frame = new JFrame("GroupLayoutExample");
6          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
7          Container contentPanel = frame.getContentPane();
8          GroupLayout groupLayout = new GroupLayout(contentPanel);
9
10         contentPanel.setLayout(groupLayout);
11
12         JLabel clickMe = new JLabel("Click Here");
13         JButton button = new JButton("This Button");
```

Group Example1:

```
20  groupLayout.setVerticalGroup(  
21  groupLayout.createParallelGroup(GroupLayout.Alignment.BASELINE)  
22      .addComponent(clickMe)  
23      .addComponent(button));  
24  frame.pack();  
25  frame.setVisible(true);  
26  }  
27  }
```


Group Example1: OUTPUT



Group Example2:

```
GroupExample2.java x GridBagLayoutDemo.java x
1  import java.awt.BorderLayout;
2  import java.awt.GridBagConstraints;
3  import java.awt.GridBagLayout;
4  import javax.swing.GroupLayout;
5  import javax.swing.JButton;
6  import javax.swing.JFrame;
7  import javax.swing.JPanel;
8
9  public class GroupExample2 {
10     public static void main(String[] args) {
11         createWindow();
12     }
13
14     private static void createWindow() {
15         JFrame frame = new JFrame("Group Layout Example2..");
16         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17         createUI(frame);
18         frame.setSize(560, 200);
19         frame.setLocationRelativeTo(null);
20         frame.setVisible(true);
21     }
```

Group Example2:

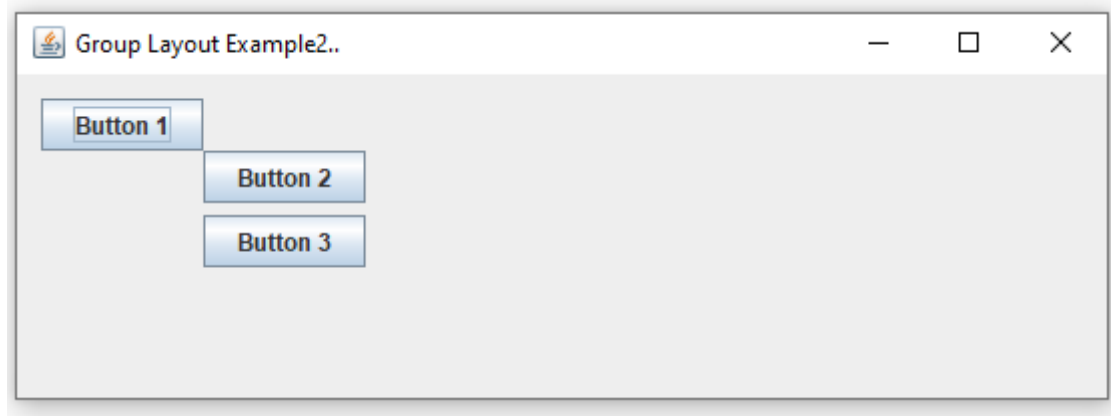
```
23 private static void createUI (final JFrame frame) {  
24     JPanel panel = new JPanel () ;  
25     GroupLayout layout = new GroupLayout (panel) ;  
26     layout.setAutoCreateGaps (true) ;  
27     layout.setAutoCreateContainerGaps (true) ;  
28  
29     JButton btn1 = new JButton ("Button 1") ;  
30     JButton btn2 = new JButton ("Button 2") ;  
31     JButton btn3 = new JButton ("Button 3") ;  
32  
33     layout.setHorizontalGroup (layout.createSequentialGroup ())  
34         .addComponent (btn1)  
35         .addGroup (layout.createSequentialGroup ())  
36         .addGroup (layout.createParallelGroup (  
37             GroupLayout.Alignment.LEADING)  
38             .addComponent (btn2)  
39             .addComponent (btn3) ) ) ) ;
```

Snipping Tool

Group Example2:

```
40
41 layout.setVerticalGroup(layout.createSequentialGroup()  
42     .addComponent(btn1)  
43     .addComponent(btn2)  
44     .addComponent(btn3));  
45  
46 panel.setLayout(layout);  
47 frame.getContentPane().add(panel, BorderLayout.CENTER);  
48 }  
49 }
```

Group Example2: OUTPUT





Spring Layout

Spring Layout

A `SpringLayout`

- *arranges the children of its associated container according to a set of constraints.*
- Constraints are nothing but horizontal and vertical distance between two-component edges.
- Every constraint is represented by a `SpringLayout`.
- Constraint object.

Each child of a `SpringLayout` container,

- as well as the container itself,
- has exactly one set of constraints associated with them.

Spring Layout

Each edge position is dependent

- on the position of the other edge.

If a constraint is added to create a new edge,

- than the previous binding is discarded.

SpringLayout doesn't automatically set the

- location of the components it manages

Spring Layout

Nested Class

Class	Description
SpringLayout.Constraints	It is a Constraints object helps to govern component's size and position change in a container that is controlled by SpringLayout

Spring Layout Fields

Fields	Descriptions
BASELINE	It specifies the baseline of a component.
EAST	It specifies the right edge of a component's bounding rectangle.
HEIGHT	It specifies the height of a component's bounding rectangle.
HORIZONTAL_CENTER	It specifies the horizontal center of a component's bounding rectangle.

Spring Layout Fields

Fields	Descriptions
NORTH	It specifies the top edge of a component's bounding rectangle.
SOUTH	It specifies the bottom edge of a component's bounding rectangle.
VERTICAL_CENTER	It specifies the vertical center of a component's bounding rectangle.
WEST	It specifies the left edge of a component's bounding rectangle.
WIDTH	It specifies the width of a component's bounding rectangle.

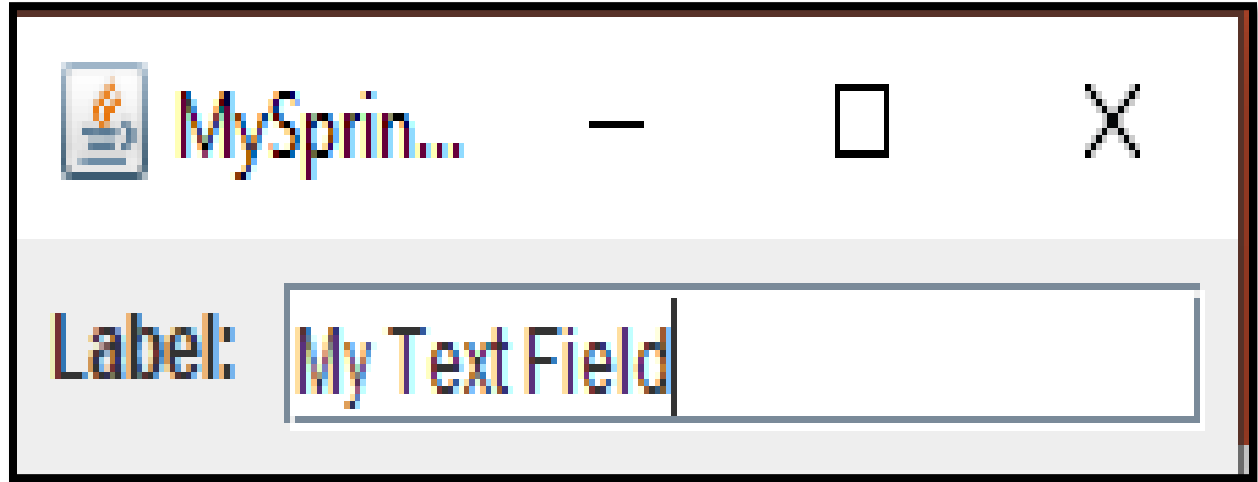
Spring Layout Example1

```
FocusListenerJavaExample.java  GridBagLayoutDemo.java  MySpringDemo.java
1  import java.awt.Container;
2  import javax.swing.*;
3  import javax.swing.SpringLayout;
4  public class MySpringDemo
5  {
6      private static void createAndShowGUI ()
7      {
8          JFrame frame = new JFrame ("MySpringDemo Example...");
9          frame.setDefaultCloseOperation (JFrame.EXIT_ON_CLOSE);
10         Container contentPane = frame.getContentPane ();
11         SpringLayout layout = new SpringLayout ();
12         contentPane.setLayout (layout);
13         JLabel label = new JLabel ("Label: ");
14         JTextField textField = new JTextField ("My Text Field", 15);
15         contentPane.add (label);
16         contentPane.add (textField);
17     }
18 }
```

Spring Layout Example1

```
18 layout.putConstraint(SpringLayout.WEST, label,6, SpringLayout.WEST, contentPane);
19 layout.putConstraint(SpringLayout.NORTH, label,6, SpringLayout.NORTH, contentPane);
20 layout.putConstraint(SpringLayout.WEST, textField,6, SpringLayout.EAST, label);
21 layout.putConstraint(SpringLayout.NORTH, textField,6, SpringLayout.NORTH, contentPane);
22 layout.putConstraint(SpringLayout.EAST, contentPane,6, SpringLayout.EAST, textField);
23 layout.putConstraint(SpringLayout.SOUTH, contentPane,6, SpringLayout.SOUTH, textField);
24
25 frame.pack();
26 frame.setVisible(true);
27 }
28 public static void main(String[] args)
29 {
30     javax.swing.SwingUtilities.invokeLater(new Runnable()
31     {
32         public void run()
33         {
34             createAndShowGUI();
35         }
36     });
37 }
38 }
```

*Spring
Layout
Example1
:Output*



*Spring
Layout
Example1
:Output*







Thank You

